Maren

SHARED WITH THE MEZO ECOSYSTEM TEAM

Security & Risk

What can go wrong in Maren, and what we do about it

DOCUMENT
MRN-004

DATE
15 August 2026

Maren Finance • Bitcoin liquidity, gasless spending, and yield • Built on Mezo Chain

DOCUMENT	DATE	CLASSIFICATION
MRN-004	15 August 2026	Shared with the Mezo ecosystem team

Security & Risk

What can go wrong in Maren, and what we do about it

CONTENTS

Summary 2

1. Risk inventory 2

2. R-1: The pooled vault 3

- The mechanism, stated honestly 3
- Why we build it anyway 3
- Mitigations 4
- The residual risk we accept 4

3. R-2: Inherited MUSD risk 5

4. R-3: Oracle 6

5. R-4 to R-9: Controls 6

- Keeper (R-4) 6
- Relayer (R-5) 6
- Contracts (R-6) 7
- Sponsorship abuse (R-7) 7
- Handles (R-8) 7
- Front-end (R-9) 7

6. Rules inherited from a real audit 7

7. Disclosure and reporting 8

Summary

This document states the risks in Maren Finance plainly, including the ones that could cost users money and the ones we have chosen to accept.

It is written on a principle borrowed from the security audit of the author's previous product: *a security document that only lists successes is not a security document.*

Three things a reader should take from it:

1. **The pooled vault is the only component that can lose user funds through a Maren design failure.** Everything else is non-custodial. We scope, cap, and audit accordingly, rather than spreading attention evenly across things that do not matter.
2. **Maren inherits risk from MUSD that we cannot mitigate, only disclose.** MUSD's contracts sit behind upgradeable proxies controlled by a single multisig, and its price oracle is a chain precompile secured by Mezo's validator set. Both are reasonable choices by Mezo. Neither is something Maren can insure against.
3. **Our controls are derived from failures that already happened**, in a payments system the same author shipped, audited on 24 July 2026, 7 Critical and 13 High findings. Each one is now a rule in this codebase.

1. Risk inventory

Ordered by expected harm, not by likelihood.

#	RISK	WHO BEARS IT	SEVERITY	STATUS
R-1	Pooled vault socialises a loss in a fast BTC drawdown	Pooled users	**Critical**	Mitigated, capped, audit scope
R-2	MUSD proxy upgrade changes behaviour underneath us	All users	**Critical**	Disclosed, not mitigable
R-3	Oracle failure or manipulation	All users	**High**	Partially mitigated
R-4	Keeper fails to act in time	Pooled users	**High**	Mitigated
R-5	Relayer key compromise	Maren treasury	Medium	Mitigated by design

#	RISK	WHO BEARS IT	SEVERITY	STATUS
R-6	Contract bug in MarenPay / Registry / Earn	Individual users	Medium	Audit + tests
R-7	Sponsorship abuse drains the gas budget	Maren treasury	Low	Mitigated
R-8	Handle squatting / impersonation in payments	Individual users	Medium	Mitigated
R-9	Front-end compromise	All users	**High**	Standard controls

2. R-1: The pooled vault

This is the risk that matters. Everything else in this document is smaller.

The mechanism, stated honestly

`MarenVault` maintains **one MUSD trove** and holds a **per-user ledger** of collateral and debt inside it. A user's collateral is **segregated in accounting, not in custody**.

That sentence is the whole risk. If the pooled trove were ever liquidated by the MUSD protocol, the loss would fall across pooled users in proportion to their share, including users whose own position was perfectly healthy.

We are not going to describe this as "your funds are safe". They are subject to a mechanism, and the mechanism can fail.

Why we build it anyway

MUSD requires **1,800 MUSD of minimum net debt**, and 2,000 MUSD of total debt once the 200 MUSD gas compensation is counted. That is a **minimum loan size**: about $3,000 of BTC to open responsibly, $5,000 to be safe, and roughly $2,200 at the 110% liquidation edge. (The $5,004.50 safe-250% figure is not an estimate: it is the output of `collateralRequiredFor()` in `packages/sdk/src/position.ts`, executed against the live oracle price, and asserted in the test suite.)

The point is not that collateral is expensive. It is that the loan has a minimum size at all, which prices small holders out entirely and pushes the few who try to the liquidation edge. Maren's pooled vault removes that minimum, safely,

which is the only honest reason to take on the risk this section describes.

Mitigations

A 30-point buffer above the cliff. The protocol liquidates at 110%. Maren liquidates internally at **140%**, and blocks further borrowing below **180%**. The pooled trove is managed to a **250% target**. A user must fall 110 percentage points below our target before the protocol has any claim on the trove.

Users who don't need pooling don't get it. `assessBorrowRoute()` sends anyone with enough collateral for a direct trove to a direct trove, which carries **no pooling risk whatsoever**, and the interface explains why. We lose pooled volume by doing this. That is the correct trade.

Borrowing pauses before withdrawal does. This is a hard requirement in the contract and has a dedicated test. If the vault must be halted, *entries* close first and *exits* stay open. A user must never be trapped in a position they want out of.

Caps that widen on evidence, not on schedule.

PHASE	CAP	GATE
Mainnet launch	5 BTC	14 days, no keeper failure
Post-audit	25 BTC	Critical/High findings resolved
Post-traction	100 BTC	60 days clean liquidation history

An insurance buffer funded from the 5% internal liquidation penalty, which absorbs rounding losses and small shortfalls before any socialisation occurs.

A minimum deposit of 0.001 BTC, which keeps positions large enough that liquidating one is economically rational for a keeper.

The residual risk we accept

A BTC drawdown faster than the keeper can act, a gap down through 140% to below 110% without an intervening block in which liquidation is possible, could socialise losses. The buffer makes this remote. It does not make it impossible.

If we cannot satisfy ourselves that this is safe, we ship without the pooled vault. Direct troves, gasless payments, and earn are valuable on their own. This commitment is in the roadmap and it is binding.

3. R-2: Inherited MUSD risk

Findings from reading the deployed contracts, offered without editorial.

MUSD's core contracts are upgradeable proxies. `PCV.owner()`, `PriceFeed.owner()`, and `MUSD.owner()` all resolve to the same Gnosis Safe, `0x98D8899c3030741925BE630C710A98B57F397C7a`. That key can upgrade implementations, swap the price oracle, and modify the MUSD mint/burn lists. `council()` and `treasury()` both currently return the zero address, so effective control sits with a Thesis/Mezo multisig rather than with tokenholders. Mezo's documentation is explicit that immutability is a later-stage goal, so this is a disclosed choice rather than an oversight.

Maren cannot mitigate this. We can only disclose it, and avoid compounding it, which is why Maren's own upgradeable contracts are behind a multisig *and* a timelock from deployment day one, rather than "before mainnet".

MUSD's backing is not what a casual reader would assume. Total supply is ~31.08M. Trove-backed debt is ~1.60M and the outstanding PCV bootstrap loan ~14.97M. The remainder sits in protocol-owned strategies and the Wormhole NTT bridge. The claim "100% Bitcoin-backed" is true of the CDP portion but **not** of most circulating MUSD today. Maren does not repeat a blanket backing claim in its interface.

Three places MUSD's documentation and its bytecode disagree. We follow the bytecode:

1. The docs state twice that a borrower with an outstanding loan pays **zero redemption fee**. `isAccountFeeExempt` appears only in `BorrowerOperations` and never in `TroveManager`, which applies `getRedemptionRate()` unconditionally. **Maren does not promise this waiver.**
2. The docs list the refinance fee as 0.1%. The code computes 20% of old net debt, then a 0.1% borrowing fee on that, approximately **0.02%**.
3. The supply/backing gap above.

These are raised with Mezo as observations, not complaints. It is entirely possible they describe intended future behaviour.

4. R-3: Oracle

MUSD's price feed reads a **native Mezo precompile** at `0x7b7c000000000000000000000000000000000015`: Mezo's own Skip Connect `x/oracle` module, fed by the validator set. It is not Chainlink and not Pyth.

Consequences we design around:

- `fetchPrice()` **reverts if the price is more than 60 seconds stale**, and it is a `view` function with **no** `lastGoodPrice()` **fallback**. Every borrow, adjust, liquidate, and redeem path can revert for reasons unrelated to the user. Both Maren apps have an explicit *price feed unavailable* state and **will not let a user borrow against a stale number**.
- The feed is secured by Mezo's validator set rather than an independent third party. This is a reasonable design for a chain that wants consensus-level prices; it is also a correlated risk, since a consensus failure is simultaneously a price failure.
- **Pyth is deployed on Mezo but its feeds run roughly 28 days stale**, and ETH/USD reverts outright. Maren never reads Pyth for liquidation math. We record this because the deployed Pyth contract looks usable and is not.

Maren's keeper treats an oracle revert as a **halt condition**, not an error to retry through: it stops opening new debt and continues to permit withdrawals.

5. R-4 to R-9: Controls

Keeper (R-4)

Runs per-block health checks, is permissionless to *call* for liquidation (so Maren is not a single point of failure), and has an economic incentive via the 5% penalty. If the keeper halts, borrowing stops; withdrawals do not.

Relayer (R-5)

A full compromise of the relayer key costs Maren its BTC gas float and nothing else. The relayer is never an owner, approver, or custodian of user funds. It relays only to allowlisted Maren contracts, only for allowlisted selectors, and derives the payer from the *recovered* signature rather than from any client-supplied field.

At Mezo's gas price, a 100,000-gas transaction costs **$0.0092**, so the float on hand is small in absolute terms even when the sponsorship budget is generous.

Contracts (R-6)

Full unit and fuzz coverage; invariant tests asserting the vault's per-user ledger cannot drift from the trove's real debt, with any rounding error required to fall in the protocol's favour; fork tests against the real MUSD contracts on matsnet. `MarenVault` and `MarenForwarder` are the audit scope.

Sponsorship abuse (R-7)

Per-address and per-IP rate limits, a global daily BTC spend cap that halts relaying, and a 25-action free tier after which a fee is taken in-band in MUSD. Exhaustion returns a **typed error**, never a silent fallback that spends our money.

Handles (R-8)

`MarenRegistry` uses commit-reveal registration to prevent front-running, restricts handles to `[a-z0-9_]` to eliminate homoglyph attacks, and the payment interface displays the resolved address alongside the handle before any confirmation. A user always sees where money is actually going.

Front-end (R-9)

Strict CSP, subresource integrity, no third-party analytics on signing paths, and transaction simulation with a human-readable summary before every signature.

---

6. Rules inherited from a real audit

The author's previous product, Talise, was audited on 24 July 2026: **7 Critical, 13 High, 20 Medium, 24 Low**. The core custody model held, the auditors found no way for one user to move another's funds, and the IDOR sweep came back clean, but the findings are now hard rules in this repository, recorded in `docs/research/SECURITY-LESSONS.md` and enforced in review.

The load-bearing ones:

- **Bind the funding source to the caller.** A relayer that accepts a client-asserted funding source can be made to fund an attacker.
- **Every value-moving parameter lives inside the signed struct**: recipient, amount, fee, deadline, nonce. Nothing economically meaningful travels beside the signature.
- **Never key escrow by a shared address.** This enabled a Critical finding where a user could fund their own claim from someone else's deposit.
- **Verify funding on-chain.** Never trust a client-supplied digest.
- **Gates fail closed.** A guard reading `if (FLAG !== "false") return true` is a pass-through whenever the variable is unset. Maren's equivalent has a test that boots with empty configuration and asserts the service refuses to operate.
- **Timelock and multisig the proxy admin at deployment**, not later.
- **A transaction hash is not success.** Only a receipt with `status === 'success'` marks a payment delivered, checked in one shared place.
- **Do not claim an attestation you have not implemented.** A control that exists in the marketing and not in the code is worse than none, because it stops people looking.
- **Audit what leaves the machine before the first mobile build.** A Critical finding was production database dumps and signing keys uploaded to a third-party build service.

The most useful thing about this list is that it is not hypothetical. Every entry is a mistake that has already been made once, by the person writing this.

---

7. Disclosure and reporting

- security@maren.finance for vulnerability reports. Never a public issue.
- Acknowledgement within 48 hours; assessment within 5 working days.
- Contracts and audit reports published in the public repository.
- Incidents disclosed publicly with a timeline, including ones caught before user harm. Talise published a six-minute production exposure and the resulting secret rotation; Maren will hold the same standard.

Maren is pre-audit. It should not be used with funds a user cannot afford to lose, and the interface says so rather than burying it in a terms page.